Go Advanced Backend Master Guide

1) 1M RPS Production-Grade Architecture in Go

- Use API Gateway (rate limiting, auth, routing).
- Horizontal scaling with Kubernetes and HPA.
- Stateless services with Redis caching.
- Postgres with read replicas and connection pooling.
- Kafka for async processing and decoupling.
- Worker pools to bound concurrency.
- Context propagation for timeouts and cancellation.
- Observability: Prometheus + Grafana + pprof.

---

2) Real pprof Analysis Walkthrough

- Enable net/http/pprof in production safely.
- Analyze CPU profile to detect hot functions.
- Use heap profile to detect memory growth.
- Check goroutine dump for leaks.
- Use flame graphs to visualize bottlenecks.

---

3) Go GC Deep Dive

- Go uses concurrent tri-color mark-and-sweep GC.
- GC triggered based on heap growth (GOGC).
- High allocations increase GC pressure.
- Reduce pointer allocations to reduce scan cost.
- Use sync.Pool to reuse temporary objects.

---

4) Zero Allocation Coding Techniques

- Preallocate slices using make with capacity.
- Reuse buffers with sync.Pool.
- Avoid fmt in hot paths; use strconv.
- Use bytes.Buffer instead of string concatenation.
- Avoid interface{} when concrete type works.

---

5) Kafka High Throughput Design in Go

- Use partitioning strategy for parallelism.
- Limit max in-flight messages.
- Batch message processing.
- Use async producer with retries.
- Implement consumer group scaling.

---

6) Leak-Free Worker Pool (Complete Code)

```
type Job struct {
    ID int
}

func worker(ctx context.Context, jobs <-chan Job, wg *sync.WaitGroup) {
    defer wg.Done()
    for {
        select {
        case job, ok := <-jobs:
            if !ok {
                return
            }
            process(job)
        case <-ctx.Done():
            return
        }
    }
}
```